Five-rule presentations of intuitionistic primitive recursive arithmetic

Closure erasure, a base-valid guard principle, and substitution

Research note prepared for Vladimir Reshetnikov

November 11, 2026

Abstract

The current `ConciseIPRA` repository already contains several substantial reductions, including a raw six-family calculus `SPrf` with substitution that preserves proof structure, and five-family clause calculi with a witness-dependent introduction. This note gives a five-family *raw* clause calculus combining the former's two-case computational equality and diagonal introduction with the latter's removal of the closure constructor. Its formulas are just equations and one two-variable binder. Its five inference families are reflexivity, evaluation, clause use, binder elimination, and binder introduction. There is no term normalization, no primitive equality-transport template, and no witness-zero mode test.

Both proof translations and both formula round trips are established. In combination with the repository's existing scalar-arithmetic bi-interpretation, the new theory is bi-interpretable and equiconsistent with full first-order iPRA, with the entire numerical domain and the identity graph in both composites. A general criterion, drawing on the earlier reflexive-base method, organizes the choice of introduction guard: preservation under substitution suffices, and preservation together with reflection gives unchanged proof-tree shape. This criterion also gives an unrestricted one-sided mirror variant.

The simplification is relative and measured. The matched complete checker is smaller than a control implementing the six-family substitution-stable interface. It is not a new overall byte minimum: the earlier witness-mode clause checker and some compact encodings are smaller. Neither global minimality nor a new Lean verification of this note is claimed.

Contents

1 Result, comparison target, and limits **3**

2 The complete presentation **3**

2.1 Raw numerical terms and formulas 3

2.2 Reusable clauses 4

2.3 The diagonal guard 4

2.4 Computational equality 4

2.5 All five inference families 5

3 Equality and structural rules **5**

4 Recovering the logical interface **6**

5 Equation induction is recovered **7**

6 All intuitionistic logical connectives **8**

7 Exact comparison with the verified native calculus **9**

8 Bi-interpretability with full first-order iPRA **10**

8.1 The interpretations as actual maps 10

9 Substitution preserves the entire proof shape **11**

10 A general base-valid guard principle **12**

10.1 The unrestricted mirror instance 13

11 The executable checker and its trust boundary **13**

11.1 Serialization and variable scope 13

11.2 Rule checking and failure behavior 14

12 Measurements and verification **14**

13 What has become simpler, and what has not **15**

14 Further research questions **16**

Reproduction and provenance **16**

1 Result, comparison target, and limits

The comparison snapshot is commit

`443cff49563457a48ee0f663012d56c4bdb58fa0`.

The source was read through the GitHub connection, including the current README, `NativeSix.lean`, `NativeDiagonal.lean`, the clause proposals, and their checkers. The last commit at this snapshot also formalizes the zero-normal successor-step interface. Thus a claim that reflexivity plus one computational step is a new reduction would be incorrect here.

The construction recommended in this note is $\mathcal{C}_\Delta$. It makes the following particular comparison:

Feature	Current raw `SPrf`	$\mathcal{C}_\Delta$
Numerical constructors	$x, 0, S, I$	$x, 0, S, I$
Formula constructors	Eq, Q, D	Eq, D
Inference families	6	5
Evaluation subcases	2	2
Term normalization	absent	absent
Supplied equality template	absent	absent
Test on the numerical witness	absent	absent
Substitution preserving proof shape	yes	yes

The reduction comes from representing a retained conditional closure as a reusable clause. Clause syntax and its variable scope are part of the specification; they are not counted as free infrastructure. The complete implementations include parsing, validation, scope checking, substitution, and the public closed-theorem interface.

This is a synthesis of two existing components, followed by an explicit equivalence and structural theorem. The clause idea occurs in `raw_clausal` and related packages; the diagonal guard and two-case evaluation are already combined in `SPrf`. The new claim is the combined five-family interface with direct proof-shape-preserving substitution. The general guard principle in Section 10 explains which additional combinations are sound.

What this does not establish. There is no meaningful proof here that five families is the smallest possible presentation, and no repository-wide checker-size record is established. Some older five-family clause checkers are smaller. The complete report therefore answers the research request with a concrete, rigorously justified simplification of the substitution-stable interface, not with an assertion that the entire existing size frontier has been beaten. All numerical comparisons identify their actual controls.

2 The complete presentation

2.1 Raw numerical terms and formulas

There is one numerical sort. Terms are finite raw syntax trees

$$t ::= x \mid 0 \mid S(t) \mid I(n, a; u.h).$$

Only u is bound in h; the counter n and initializer a are outside that binder. The update h may use passive parameters. Every substitution below is ordinary capture-avoiding substitution, including beneath nested iterator bodies. In particular,

$$I(0, a; u.h) \not\equiv a$$

as syntax. Their equality will be proved. The symbol $\equiv$ always means literal syntax equality, up to the fixed representation of bound variables.

Formulas have two constructors:

$$A ::= (s = t) \mid D_{x,y}A.$$

Both x and y are bound in $D_{x,y}A$. The numerical term $S0$ is abbreviated 1. The designated contradiction is $0 = 1$.

For explanation and for the eventual interpretation, the ordinary first-order reading is

$$(D_{x,y}B)^\circ := \exists x(B^\circ(x, 0) \to \forall y\, B^\circ(x, y)). \tag{1}$$

This reading is not an additional inference rule.

2.2 Reusable clauses

A context is a finite list of clause *occurrences*

$$[A; y.B(y)], \qquad y \notin \mathrm{FV}(A).$$

The displayed y is bound only in the consequent B. The ordinary reading is $A^\circ \to \forall y B^\circ(y)$. Two occurrences with identical values can have different positions and different origins.

Let $H := (0 = 0)$. An ordinary retained assumption F abbreviates the constant clause

$$\mathrm{assm}(F) := [H; v.F], \qquad v \notin \mathrm{FV}(F). \tag{2}$$

Under de Bruijn notation the body is weakened under its unused binder. With the implementation's locally nameless levels, a locally closed F can be inserted unchanged. Writing Γ, F is only shorthand for adjoining this clause.

2.3 The diagonal guard

The raw matrix $B(x, y)$ is diagonal when it has the literal shape

$$I(y, y; u.h) = I(y, y; v.k). \tag{3}$$

All four displayed y occurrences refer to the inner variable of the enclosing D. The update terms h, k are unrestricted. The test is made on the matrix *before* the witness is substituted and before y is opened. Define

$$\eta_B(z) = \begin{cases} z, & \mathrm{diag}_y(B), \\ 0, & \text{otherwise.} \end{cases}$$

2.4 Computational equality

The partial operation $\Phi(s, t, b)$ has exactly the following cases:

Shape of b	Required literal source s	Result $\Phi(s, t, b)$
$I(0, a; u.h)$	a	t
$I(Sc, a; u.h)$	$I(c, a; u.h)$	$h[t/u]$

It is undefined in every other case. A variable or a numeral merely provably equal to a successor does not satisfy the literal successor test. The two rows are disjoint. There is no third equality-composition case.

2.5 All five inference families

These are the entire primitive calculus:

$$\frac{}{\Gamma \vdash t = t} \tag{R}$$

$$\frac{\Gamma \vdash s = t \qquad \Gamma \vdash b = r}{\Gamma \vdash r = \Phi(s,t,b)} \tag{V}$$

$$\frac{\Gamma \vdash A}{\Gamma \vdash B(q)} \quad [A; y.B] \text{ is retained in } \Gamma \tag{U}$$

$$\frac{\Gamma \vdash D_{x,y}B \qquad \Gamma, [B(b,0); y.B(b,y)] \vdash A}{\Gamma \vdash A} \tag{E}$$

$$\frac{\Gamma, \mathrm{assm}(B(t, \eta_B(z))) \vdash B(t, Sz)}{\Gamma \vdash D_{x,y}B} \tag{I}.$$

V requires Φ to be defined. In E, b is fresh for the outer context, the conclusion, and the passive parameters of B. In I, z is fresh for the outer context, the witness t, and the passive parameters. Contexts are weakened into the new scope. All syntax is well formed and scoped.

There are five inference *families*, with two subcases in V. If those subcases are counted as separate constructors, the count is six, just as the repository's six-family `SPrf` has seven Lean constructors. No advantage is claimed by changing this counting convention. Hypothesis, weakening, cut, substitution, equality transport, both iterator equations, numerical cases, and equation induction are derived, not additional primitives.

3 Equality and structural rules

Lemma 3.1 (Ordinary hypotheses, weakening, and ordinary cut)**.** *An ordinary retained assumption can be used. Weakening and fresh renaming are admissible. If* $\Gamma \vdash F$ *and* $\Gamma, \mathrm{assm}(F) \vdash A$*, then* $\Gamma \vdash A$*.*

Proof. R at 0 proves H; U on (2), with any target, proves F. Weakening and injective fresh renaming are inductions on the derivation. The diagonal test is invariant under these renamings.

For cut, distinguish the occurrence of $\mathrm{assm}(F)$. Replace every U selecting that occurrence by the supplied proof of F, weakened into the local scope. Its proof of H can be discarded. Other nodes are translated recursively. Freshen eigenvariables before inserting proofs. The construction is sensitive to clause occurrences, so equal-looking clauses cause no ambiguity. It does not assume substitution of arbitrary numerical terms into derivations. □

Lemma 3.2 (Root composition from two zero evaluations)**.** *From* $s = t$ *and* $s = r$ *one can derive* $r = t$ *using only R and V.*

Proof. Set $W_s = I(0, s; u.u)$. Apply the zero row of V to $s = r$ and $W_s = W_s$, obtaining $W_s = r$. Apply it again to $s = t$ and $W_s = r$, obtaining $r = t$. □

Write $\mathrm{pivot}(p, q)$ for this derived operation. Symmetry sends a proof $p : s = t$ to $\mathrm{pivot}(s = s, p) : t = s$. If $p : s = t$ and $q : t = v$, then $\mathrm{pivot}(q, \mathrm{sym}(p))$ proves $s = v$.

Lemma 3.3 (Computation and arbitrary term congruence)**.** *The calculus proves*

$$I(0,a;u.h) = a, \qquad I(Sc,a;u.h) = h[I(c,a;u.h)/u].$$

For every fixed term template $h(u)$, $s = t$ entails $h(s) = h(t)$.

Proof. Each computation equation is one V node on two appropriate reflexivities. For congruence, put $A_s = I(0,s;u.h)$ and $B_s = I(S0,s;u.h)$. From $p : s = t$ construct

$$\begin{aligned} A_s &= s && \text{by zero evaluation,}\\ B_s &= h(s) && \text{by successor evaluation using the preceding line,}\\ A_s &= t && \text{by zero evaluation using } p,\\ h(s) &= h(t) && \text{by successor evaluation using the last two lines.} \end{aligned}$$

The last application has first source exactly A_s and second source exactly B_s. Thus the argument does not use a pre-existing congruence rule. Nested occurrences of u are handled by the ordinary capture-avoiding substitution already used in V. □

Combining congruence with Lemma 3.2 gives one-term crossed transport

$$s = t,\ h(s) = r \quad \Longrightarrow \quad r = h(t).$$

Applying it twice gives transport in any equation template $a(s) = b(s)$. Both results hold at all sources, including literal 0, with no induction.

4 Recovering the logical interface

Lemma 4.1 (The guarded base is a theorem)**.** *If* $\mathrm{diag}_y(B)$*, then* $\Gamma \vdash B(t,0)$ *for every witness t and every context Γ.*

Proof. The instance has the form $I(0,0;u.h') = I(0,0;v.k')$. Both sides compute to 0; symmetry and transitivity join them. Neither update body is evaluated. □

Lemma 4.2 (Every case introduction)**.** *For every matrix B and every t, the following rule is admissible:*

$$\frac{\Gamma, \mathrm{assm}(B(t,0)) \vdash B(t,Sz)}{\Gamma \vdash D_{x,y}B}.$$

Proof. For a nondiagonal matrix this is primitive I. For a diagonal matrix cut the theorem $B(t,0)$ from Lemma 4.1 into the supplied case branch. The result proves $B(t,Sz)$ without that local assumption. Weaken by $\mathrm{assm}(B(t,z))$ and use primitive I in progress mode. □

Define a closure *abbreviation*, using an unused w,

$$\mathsf{q}_y B(y) := D_{w,y}B(y), \qquad w \notin \mathrm{FV}(B). \tag{4}$$

No Q node is added to the public syntax.

Lemma 4.3 (Closure interfaces)**.** *The following operations are derivable:*

1. $\Gamma, B(0) \vdash B(Sz)$ *entails* $\Gamma \vdash \mathsf{q}_y B$.
2. $\Gamma \vdash \mathsf{q}_y B$ *and* $\Gamma \vdash B(0)$ *entail* $\Gamma \vdash B(t)$.
3. *A retained clause* $[B(0); y.B]$ *entails* $\mathsf{q}_y B$.

Proof. The first is Lemma 4.2 with an unused witness. For the second, E opens the dummy-witness D, yielding the displayed canonical clause; U applies it to the supplied base proof. For the third, under the local assumption $B(0)$, use the retained clause at Sz, then apply the first operation. This argument is also valid when the matrix is diagonal, because all case introductions have already been recovered. □

Consequently numerical case distinction is available for every formula: proofs of $A(0)$ and $A(Sz)$ for fresh z give a proof of $A(t)$ at any target. Use the successor proof in the first closure interface and the zero proof in the second. No induction is used.

Lemma 4.4 (Transport in arbitrary formulas)**.** *For every Eq/D template $F(v)$, $s = t$ and $F(s)$ entail $F(t)$.*

Proof. Proceed by structural induction on F. The equation case was proved above. For $F(v) = D_{x,y}B(v, x, y)$, eliminate $F(s)$ at fresh b to retain $[B(s, b, 0); y.B(s, b, y)]$. Introduce $F(t)$ by case introduction with witness b. Its local assumption $B(t, b, 0)$ transports backwards by the induction hypothesis. U produces $B(s, b, Sz)$; transport forwards gives $B(t, b, Sz)$. Close I and E. Only the strictly smaller matrix uses the induction hypothesis. This proof uses no arithmetic induction. □

5 Equation induction is recovered

We give an explicit proof so that the diagonal guard does not conceal a missing arithmetic principle. Put

$$C(v, a, b) := I(v, a; u.b),$$

with u unused in b. Computation gives $C(0, a, b) = a$ and $C(Sn, a, b) = b$.

For an arbitrary equation $E(y) \equiv s(y) = t(y)$ define

$$H_E(y) := L_s(y) = L_t(y), \qquad L_s(y) := I(y, y; u.s(y)), \quad L_t(y) := I(y, y; u.t(y)), \tag{5}$$

where the update variable is unused in s, t. The lift is diagonal, its zero instance is a theorem, and computation gives

$$H_E(Sz) \;\dashv\vdash\; E(Sz). \tag{6}$$

Numerical cases prove, without induction, the unconditional equations

$$C(y, s(0), L_s(y)) = s(y), \qquad C(y, t(0), L_t(y)) = t(y). \tag{7}$$

At 0 these are reflexive after computation; at a successor the inner and outer constant-update iterators compute to the displayed right side. Hence, under $E(0)$, congruence and (7) give

$$E(0), H_E(y) \quad \vdash \quad E(y). \tag{8}$$

Theorem 5.1 (Contextual equation induction)**.** *For every literal equation $E(y)$, every context Γ, and every numerical term q, proofs of*

$$\Gamma \vdash E(0), \qquad \Gamma, \mathrm{assm}(E(z)) \vdash E(Sz)$$

with fresh z effectively yield $\Gamma \vdash E(q)$.

Proof. Under the local assumption $H_E(z)$, recover $E(z)$ by (8). Cut the given step proof at its original fresh variable, obtaining $E(Sz)$, then use (6) to obtain $H_E(Sz)$. Primitive I with a dummy witness now proves $\mathsf{q}_y H_E$. Its zero instance is a theorem, so closure elimination gives $H_E(q)$. Apply (8) once more.

No substitution is made into the supplied step proof. The equality induction being derived is not used to prove the selector identities. This makes the dependency order explicit. □

The usual progress introduction $E(z) \vdash E(Sz) \Rightarrow \mathsf{q}_y E(y)$ also follows. Temporarily retain $E(0)$, perform the construction to obtain $E(Sw)$ at a fresh target, then close case introduction for $\mathsf{q}_y E$. With arbitrary passive witness parameters, this recovers witnessed progress introduction as well.

6 All intuitionistic logical connectives

The absence of primitive connective symbols does not mean that only an equational or positive fragment is represented. The following definitions expand to Eq/D formulas. The construction is the native selector construction with every Q replaced by (4).

First define a selector between an equation and an arbitrary formula:

$$\begin{aligned} \mathrm{esel}_v(s,t;r=q) &:= (C(v,s,r) = C(v,t,q)),\\ \mathrm{esel}_v(s,t;D_{x,y}B) &:= D_{x,y}\,\mathrm{esel}_v(C(y,0,s),C(y,0,t);B). \end{aligned}$$

All bound names are freshened away from v, s, t. Put $\mathrm{gate}_v(A) := \mathrm{esel}_v(0,0;A)$. Recursively define a selector between any two formulas:

$$\begin{aligned} \mathrm{sel}_v(s=t,B) &:= \mathrm{esel}_v(s,t;B),\\ \mathrm{sel}_v(D_{x,y}A,B) &:= D_{x,y}\,\mathrm{sel}_v(A,\mathrm{gate}_y(B)). \end{aligned}$$

Again the new binders are fresh for v and B. Structural induction, computation, numerical cases, and the closure interfaces prove

$$\mathrm{sel}_0(A,B) \dashv\vdash A, \quad \mathrm{sel}_{Sz}(A,B) \dashv\vdash B, \quad \mathrm{gate}_0(A) \dashv\vdash H, \quad \mathrm{gate}_{Sz}(A) \dashv\vdash A.$$

For example, the binder step for an equation selector reduces at $v = 0$ to $D_{x,y}(C(y,0,s) = C(y,0,t))$, whose base is true and whose successor instance is $s = t$. It is therefore equivalent to $s = t$.

Let $\mathrm{par}(y) = I(y,0;u.C(u,1,0))$. The logical abbreviations are

$$\begin{aligned} \bot &:= (0=1),\\ A \to B &:= \mathsf{q}_y\,\mathrm{sel}_y(A,B),\\ A \wedge B &:= \mathsf{q}_y\,\mathrm{gate}_y(\mathrm{sel}_{\mathrm{par}(y)}(A,B)),\\ \exists x\,A(x) &:= D_{x,y}\,\mathrm{gate}_y(A(x)),\\ A \vee B &:= \exists v\,\mathrm{sel}_v(A,B),\\ \forall x\,A(x) &:= A(0) \wedge \mathsf{q}_x A(x). \end{aligned}$$

Every unused variable is fresh. The existential notation on the right side of the disjunction definition is the preceding macro.

These have their usual intuitionistic introduction and elimination rules. Implication follows because the zero selector instance is A and every successor instance is B. The conjunction gate

has true zero base; its instances at 1 and 2 recover B and A. Conversely, A and B imply every selector instance by numerical cases. The existential gate similarly has true zero base and its instance at 1 recovers the witness formula. Universal quantification is its zero instance together with conditional closure. These are the same explicit native macros whose rule derivations are recorded in `NativeLogic.lean` [6].

Finally, $0 = 1$ implies every equation $s = t$: transport through $C(v, s, t)$ and compute its two endpoints. Structural induction on Eq/D formulas, using case introduction with witness 0, gives ex falso for every formula. No law of excluded middle for arbitrary formulas is assumed or derived by the presentation.

7 Exact comparison with the verified native calculus

Let $\mathcal{S}_6$ denote the repository's `SPrf`: Eq/Q/D formulas, ordinary formula contexts, and Hyp, R, the same two-case V, QE, DE, and the same diagonal introduction. The source proves

$$\texttt{sprf_iff_nprf}: \quad \texttt{SPrf}\,\Gamma\,A \longleftrightarrow \texttt{NPrf}\,\Gamma\,A.$$

This theorem is part of the pinned repository, not a new machine-checked claim about the present checker [2].

Define j to include Eq/D syntax into native syntax, and define τ by

$$\begin{aligned} \tau(s = t) &:= s = t, \\ \tau(D_{x,y}B) &:= D_{x,y}\tau(B), \\ \tau(Q_yB) &:= D_{w,y}\tau(B), \qquad w \notin \mathrm{FV}(B). \end{aligned}$$

Terms are unchanged. The translation commutes with capture-avoiding numerical substitution and preserves and reflects the diagonal predicate on matrices: an equation is unchanged in shape, and a compound formula stays compound.

Theorem 7.1 (Proof translations)**.** *For every native context* Γ *and formula* A*,*

$$\Gamma \vdash_{\mathcal{S}_6} A \quad \Longrightarrow \quad \mathrm{assm}(\tau\Gamma) \vdash_{\mathcal{C}_\Delta} \tau A.$$

For every Eq/D context Δ *and formula* F*,*

$$\mathrm{assm}(\Delta) \vdash_{\mathcal{C}_\Delta} F \quad \Longleftrightarrow \quad j\Delta \vdash_{\mathcal{S}_6} jF.$$

Proof. For the first implication induct on the native proof. Hyp becomes R at 0 followed by U on the corresponding constant clause. R and V are copied. QE becomes closure elimination from Lemma 4.3. I is copied because τ preserves the diagonal guard and the witness.

For native DE, translate its major and use E to retain the canonical clause $[\tau B(b, 0); y.\tau B(b, y)]$. Lemma 4.3 closes this clause to $\mathtt{q}_y\tau B(b, y)$, precisely the translation of the old Q assumption. Cut that proof into the translated old branch. Close E. The dummy witness and the branch eigenvariable are freshened as needed.

For the reverse implication annotate every clause by its origin. An initial ordinary clause or I-branch clause is represented by its ordinary formula assumption in $\mathcal{S}_6$. An E-branch clause is represented by the native assumption $Q_yjB(b, y)$. A U selecting the first kind becomes Hyp; a U selecting the second becomes QE applied to the translated antecedent proof. R and V are copied; E and I become native DE and DI. The annotation belongs to the occurrence, so identical-looking clauses of different origins are handled correctly. This is an induction on the clause derivation. The forward implication on the common language is the first statement because τj is the identity. □

For arbitrary externally supplied clauses, their ordinary readings can instead be represented by the implication and universal macros of Section 6. U then translates by implication and universal elimination. The common-language theorem already covers every finite ordinary-assumption sequent, not only closed theorems.

Theorem 7.2 (Both formula round trips)**.** *For every Eq/D formula F, $\tau j F = F$ literally up to bound renaming. For every native formula A,*

$$A \dashv\vdash_{\mathcal{S}_6} j\tau A.$$

Both numerical translations are the identity.

Proof. The only nontrivial atomic step in the structural argument is

$$Q_y B \dashv\vdash D_{w,y} B, \qquad w \notin \mathrm{FV}(B).$$

From the left, use QE under the local zero assumption and then case introduction with a dummy witness. From the right, native DE opens exactly the Q assumption required for the conclusion.

Uniform pointwise interderivability lifts through D: eliminate the source binder, introduce the target by cases, convert its zero assumption backwards, use the retained source closure at a successor, and convert forwards. It lifts through Q by the same construction with a dummy witness. Structural induction proves the stated result. This establishes formula round trips, not merely mutual consistency. □

8 Bi-interpretability with full first-order iPRA

The target iPRA is the theory formalized in `PR.lean`: intuitionistic first-order logic with equality, one symbol for every primitive-recursive definition, its defining equations, successor injectivity and separation, and induction for quantifier-free formulas. This specifies the target precisely; the result is not only about its universal equational fragment.

The repository supplies a verified intermediate theory T_{eq}. It uses the raw scalar terms under ordinary intuitionistic logic, the two iterator equations, numerical coverage

$$x = 0 \vee \exists y\, x = Sy,$$

separation $0 = S0 \to \bot$, and induction for literal equations. It proves both directions and both composites between this theory and iPRA, then between the native calculus and T_{eq} [8, 7].

8.1 The interpretations as actual maps

In the direction from $\mathcal{C}_\Delta$ to iPRA, interpret D by (1); equality is actual equality. Compile every scalar term recursively. For a fixed update template $h(u, \vec{p})$, use the primitive-recursive function defined by

$$f_h(0, a, \vec{p}) = a, \qquad f_h(Sn, a, \vec{p}) = h(f_h(n, a, \vec{p}), \vec{p}).$$

All free parameters of the update are included in $\vec{p}$, even if one also occurs in the counter expression. Thus no universal primitive-recursive evaluator is postulated.

In the reverse direction use the repository's scalar compilation for primitive-recursive symbols, followed by the Eq/D logical macros. The main primitive-recursion compilation has the explicit form

$$F(n, \vec{p}) = \pi_2 I\,(n, \langle 0, g(\vec{p})\rangle; z.\, \langle S\pi_1 z,\ h(\pi_1 z, \pi_2 z, \vec{p})\rangle) \tag{9}$$

for $F(0,\vec{p}) = g(\vec{p})$ and $F(Sn,\vec{p}) = h(n,F(n,\vec{p}),\vec{p})$. Here the pairing and projection terms are the derived scalar terms verified in `TeqPair.lean`; they are macros, not new primitives. Composition is term substitution, and zero, successor, and projections translate directly. The existing arithmetic bridge proves the required pairing equations, quantifier-free reflection, and uniqueness needed for the round trips. This note reuses that bridge rather than purporting to reverify it with finite Python tests.

Theorem 8.1 (Bi-interpretability and equiconsistency)**.** *The ordinary realization of $\mathcal{C}_\Delta$ is bi-interpretable with full first-order* iPRA*. The numerical domains in both interpretations are the whole domains, equality is actual equality, and the numerical graph of both composites is $x = y$. In particular,*

$$\mathcal{C}_\Delta \vdash 0 = S0 \quad \Longleftrightarrow \quad \mathsf{iPRA} \vdash \bot.$$

Proof. Compose Theorems 7.1 and 7.2 with the pinned native–T_{eq}–iPRA bi-interpretation. The new maps leave numerical terms unchanged before the existing scalar translation, and D retains exactly its native reading. One new formula composite is literal identity; the other is the provable equivalence $Q \leftrightarrow D$ with a dummy witness. The old composites are already internally provable identities. Composing these equivalences gives the required internal formula composites, with identity numerical graph. No proper numerical subdomain, quotient, nonstandard coding domain, or restriction to standard models is used.

The contradiction translation is the existing native contradiction translation. Proof preservation in both directions therefore gives the final equivalence. □

The logical dependency on the existing verified bridge is explicit. The new proof translation and guard metatheorems are written proofs in this report. No Lean executable was available in this run, so the archive does not claim a newly compiled Lean proof of them.

9 Substitution preserves the entire proof shape

Lemma 9.1 (Preservation and reflection of the diagonal shape)**.** *Let σ substitute terms for passive variables, lifted under the two variables of D. Then*

$$\mathrm{diag}_y(B\sigma) \quad \Longleftrightarrow \quad \mathrm{diag}_y(B).$$

Proof. Substitution cannot change an existing iterator constructor, and it fixes the bound y. It cannot turn a different bound variable into y. A substituted passive-variable term is capture-free and therefore has no occurrence of the surrounding bound y. Such a term cannot be the variable y, nor can it be an iterator with counter and initializer both y. Thus neither of the two required iterator heads can be manufactured by substituting a passive variable. The statement follows for both sides of the equation; compound formula roots remain compound. □

Theorem 9.2 (Proof-shape-preserving numerical substitution)**.** *If $\Gamma \vdash_{\mathcal{C}_\Delta} A$ and σ is any well-scoped capture-avoiding numerical substitution, then $\Gamma\sigma \vdash_{\mathcal{C}_\Delta} A\sigma$. A derivation is obtained with the same inference-family tree, the same clause-occurrence selections, and the same evaluation subcases, after fresh eigenvariables are consistently renamed.*

Proof. Induct directly on the proof. R is immediate. Each V source identity is preserved by substitution, and capture-avoiding substitution composes with update-body instantiation. U preserves the selected clause occurrence and its antecedent identity. E uses a fresh variable in the substituted context. For I, Lemma 9.1 fixes whether its local assumption is the zero

instance or the progress instance. Its substituted branch is therefore exactly the branch required by the same I node. There are no additional proof nodes. □

The existing witness-mode clause calculus also has substitution *admissibility*, obtainable through its equivalence with the native calculus. The distinction is that its literal test "the witness is 0" may change when a variable is substituted by 0. Its proof cannot in general be transformed by copying every local rule instance. The result above concerns the stronger, directly checkable structural property, not a claim that the old system lacks substitution altogether.

This matters when the presentation is used as an object language inside a proof assistant: substitution, renaming, and subsequent compiler-correctness arguments can share a structural induction without repairing introduction branches. It does not by itself imply a smaller standalone checker than every witness-mode implementation.

10 A general base-valid guard principle

There is a useful generalization behind the construction. The one-way preservation and local-repair idea is already explicit in the filed reflexive-base proposal [5]; the following criterion packages that method with closure erasure and the equation-lift requirement. Let $G_y(B)$ be a decidable raw-syntax predicate on matrices, invariant under alpha renaming and injective fresh renaming of passive variables. Replace the diagonal predicate in I by G, leaving every other definition unchanged. Call the result $\mathcal{C}_G$. Assume the following properties:

(a) Every guarded matrix is a literal equation.

(b) Its zero instance $B(t, 0)$ is uniformly derivable using R and V only.

(c) For every equation $E(y)$ there is a guarded lift $H_E(y)$ with theorem $H_E(0)$, and derivations

$$H_E(Sz) \dashv\vdash E(Sz), \qquad E(0), H_E(y) \vdash E(y),$$

using equality reasoning and numerical cases only.

The last requirement is an effective construction, not a semantic promise about standard natural numbers.

Theorem 10.1 (Guard criterion)**.** *Under (a)–(c), $\mathcal{C}_G$ is proof-equivalent on its common language to the native scalar core, and is bi-interpretable with* iPRA*. If in addition G is preserved by capture-avoiding substitution of passive variables, substitution is admissible by a direct proof transformation. If G is also reflected by substitution, that transformation preserves the proof tree exactly. Without reflection, a local repair can be needed.*

Proof. Property (b) recovers all case introductions exactly as in Lemma 4.2. The closure interfaces, numerical cases, and formula transport then follow before equation induction. Property (c) recovers arbitrary equation induction exactly as in Theorem 5.1. Every primitive guarded I is sound in the native calculus because (a) makes it an ordinary equation-progress instance; every unguarded I is a case instance. Conversely, all native introduction and equality interfaces have now been derived. Closure erasure and the same two formula composites give the equivalence.

For substitution use a direct induction. A guarded matrix stays guarded by the extra preservation assumption. A nondiagonal mode that remains unguarded is also immediate. The only new possibility is an originally unguarded matrix becoming guarded. The inductively

substituted branch assumes the substituted zero instance. Property (b) supplies a proof of that base. Cut it into the branch, weaken by the substituted progress instance, and apply guarded I. This is a finite local proof transformation using ordinary cut. If G also reflects substitution, that exceptional case cannot occur, and no proof nodes are inserted. □

10.1 The unrestricted mirror instance

A simpler-looking, one-sided guard is

$$\mathrm{mirror}_y(B) \quad \Longleftrightarrow \quad B \equiv a = I(y, a; u.h). \tag{10}$$

There is no restriction on the public Eq/D grammar: every other matrix uses the case branch. This differs from the previously filed mirror proposal's guarded public-language presentation. The guard and its basic lift themselves are inherited ideas; the unrestricted dispatch and its position in the guard criterion are the refinement here.

Property (b) is the symmetry of zero computation. For $E(y) : s(y) = t(y)$ use

$$M_E(y) := s(y) = I(y, s(y); u.t(y)).$$

The update variable is unused. The successor instance computes to $E(Sz)$. Numerical cases prove

$$C(y, s(0), s(y)) = s(y), \qquad C(y, t(0), I(y, s(y); u.t(y))) = t(y).$$

Under $E(0)$ and $M_E(y)$, congruence joins the two left sides; hence $E(y)$. This verifies (c) without assuming induction.

The mirror guard is preserved by passive substitution, but need not be reflected. For distinct passive variables p, q, the matrix

$$p = I(y, q; u.h)$$

is not mirror-guarded, but a substitution identifying p and q makes it so. Theorem 10.1 handles this change by proving and cutting the now-trivial zero base. Thus the mirror calculus $\mathcal{C}_{\mathrm{M}}$ is a valid alternative, though it lacks $\mathcal{C}_{\Delta}$'s exact proof-shape theorem. The archive keeps this distinction visible.

11 The executable checker and its trust boundary

11.1 Serialization and variable scope

The standalone checker uses JSON and Python's standard library. Numerical zero is integer 0. A successor is `[0,t]`; an iterator is `[n,a,h]`. A formula is `[s,t]` for equality and `[B]` for D.

Positive integers are bound de Bruijn indices: 1 denotes the nearest binder. Negative integers name local free levels. At local scope k and binder depth d, a numerical leaf must satisfy $-k \le a \le d$. Iteration adds one binder in its third child. A D matrix adds two. Opening the outer D witness removes index 2 while keeping its inner control index 1. Inserted terms are locally closed with respect to external positive indices: free parameters use negative levels. Consequently the opener need not shift an inserted value under surrounding binders; its internal bound indices remain internal. Branch scope introduces a new negative level $-k - 1$.

The diagonal test on a validated matrix is exactly that both equation sides have iterator prefix `[1,1]`. The shape is inspected before the witness is opened. This is a structural binding test, not a comparison of numerical values.

The public input has exactly two keys, `goal` and `proof`. The goal must be a closed Eq/D formula. Proof tags are `r,u,v,e,i`, with exact arities. Only genuine Python integers and lists are allowed as syntax; Boolean and floating-point aliases are rejected. Source and target scope checks are included. An elimination branch must return a conclusion already valid in the outer scope, preventing eigenvariable escape.

11.2 Rule checking and failure behavior

The checker reconstructs the conclusion of each proof node. It does not accept a supplied equation merely because that equation is true on tested numerals. V decomposes the two derived equation premises, checks the literal iterator source, and performs one body opening. The validity invariant ensures that term and formula roles cannot be exchanged in a successful derivation.

Malformed structures, failed source matches, and out-of-scope variables reject. The driver prints `ACCEPT` and exits successfully only after the reconstructed result equals the validated closed goal. It uses explicit checks, not Python `assert`, so optimized execution with `-O` has the same acceptance behavior on the tested interface. As with ordinary Python recursive implementations, finite resource exhaustion can prevent completion on very large certificates; it does not provide an acceptance path.

The following are untrusted conveniences: proof constructors, test generators, comparison tools, and the translator into the native control. They are not imported by the standalone checker. Their purpose is to produce auditable examples and to expose implementation mistakes. The executable translator handles closed primary certificates and the two clause origins generated by the public checker; the more general correspondence for arbitrary contexts and the reverse direction are mathematical proofs, not an additional implemented compiler.

12 Measurements and verification

The attached machine-readable results and reproduction script report the executed measurements. The final complete-module measurements use CPython 3.12.14, removal of actual module/function/class docstrings, `ast.unparse` normalization, and one final line feed:

Complete program	Bytes	Tokens	AST nodes
New diagonal clause checker	3056	872	1013
Matched native diagonal control	3395	975	1138
Existing witness-mode clause control	2956	837	977

Documentation strings are excluded identically in all three columns; the evidence separately records raw source sizes. Descriptive titles are therefore not credited as mathematical improvements. Common functions and variables retain the same names in the matched family. The matched native control retains the same numerical representation and equality code, but has a third formula constructor and separate hypothesis and closure-elimination interfaces.

The complete-module reduction against that control is 339 bytes, approximately 10.0%; tokens and AST nodes also decrease. The earlier witness-mode clause checker is 100 bytes smaller than the new checker. The optional mirror checker measures 3,059 bytes, 874 tokens, and 1,012 AST nodes: three more bytes and two more tokens than the primary checker, with one fewer AST node. It is included for its structural comparison, not as a byte improvement. The repository additionally contains compact encodings, including a reported 516-byte core and an

860-byte validated Eq/Q/D program under their respective contracts. Their tiny sizes must not be presented as directly comparable to the readable, independently structured reference checker in this archive. In particular, this note has not beaten those minima.

The executed verification includes:

- Accepted certificates for both iterator computations, root composition, symmetry, nested-template congruence, genuine use of the induction hypothesis, nested binders, repeated clause use, and elimination using its opened clause.
- Translation of every positive primary fixture into the matched native diagonal control, with exact equality of the reconstructed final formula.
- Negative fixtures for malformed terms and tags, extra fields, Boolean and floating-point syntax, wrong evaluation sources, a nonliteral successor counter, invalid clause indices and antecedents, unbound witnesses, and eigenvariable escape.
- Numerical substitutions including a variable witness becoming zero, consistent eigenvariable renaming, and substitutions inside nested iterator bodies.
- Guard-invariance and manually specified capture-avoidance checks, together with normal and optimized public-driver runs.

The primary run passed 12 positive proofs, 17 rejection fixtures, 12 exact primary-to-native translations, 5 parameter substitutions, 600 guard cases, 4 explicit opening controls, and 82 public process checks. The mirror run passed 13 positive and 13 negative certificates, 300 guard-preservation cases, 52 process checks, and 5 explicit guard-creation substitutions whose naive proof images fail and whose repaired images succeed. The progressive-hypothesis fixture has a reflexive conclusion: it exercises genuine local hypothesis use and rule-mode preservation, not a new deep induction theorem.

The evidence distinguishes finite tests from proofs. These tests check the executable implementation and translator on the stated families. They do not prove universal soundness, completeness, or bi-interpretability; those claims rest on the written derivations and the explicitly cited existing arithmetic bridge.

13 What has become simpler, and what has not

Three parts of the specification become smaller together: there is no closure formula constructor, no independent hypothesis instruction, and no separate closure-elimination instruction. Clause use covers ordinary assumptions and opened conditional closures, while a dummy witness supplies a closure whenever one must be concluded as a formula. The diagonal rule makes this elimination of Q compatible with direct, shape-preserving substitution.

The arithmetic base is unchanged. R plus the two rows of V already appeared in the current repository, and the diagonal guard is already formalized there. Presenting these as a new arithmetic breakthrough would misstate the result. Likewise, the mirror guard appeared previously in a restricted-language proposal. The general guard theorem explains why that language restriction can be dropped when unguarded matrices take the case branch, and why failure of guard reflection does not destroy substitution admissibility.

A global simplicity order would need to fix more than the number of displayed rules. It would need a common syntax-validation contract, an encoding convention, the treatment of normalization and parser libraries, and a size metric that does not reward arbitrary name shortening. This report keeps its actual byte comparison separate from its mathematical rule

comparison. It supplies a usable smaller interface for one important baseline, not a proof that every other presentation is worse.

14 Further research questions

1. **Formalize clause origins directly.** A Lean embedding of $\mathcal{C}_\Delta$ could use a separate indexed clause context and a derivation relation. Its substitution theorem should then be one structural induction, while its equivalence proof can reuse `SPrf`. The occurrence annotations in the reverse compiler should be represented explicitly.
2. **Find a smaller guard satisfying the criterion.** A proposed predicate must have a uniformly provable zero base and an effective guarded lift of every equation. Merely checking one iterator side, without its initializer relation, does not establish the base condition.
3. **Compare proof size as well as checker size.** Closing and reopening a dummy-witness closure can increase certificate length. The present proof translations are effective, but this note does not assert a uniform linear serialized-size bound. Sharing and ordinary-cut expansion require separate analysis.
4. **Use the guard criterion to compare repairs.** The mirror variant can require a cut repair after substitution; the diagonal variant cannot. The costs of those repairs in proof-assistant definitions and generated certificates can be measured independently of checker bytes.
5. **Normalize comparisons before optimizing.** Reimplement the smallest existing protocols and the new one under a single exact malformed-input policy, a common naming convention, and an agreed documentation-free metric. That would make a further record claim meaningful.
6. **Investigate a genuine arithmetic reduction.** Deleting reflexivity or replacing it with a finite set of closed true equality seeds is already obstructed in the filed clause work. A further reduction is likely to require a different equality interface or term language, with its own explicit whole-domain round trips.

Reproduction and provenance

The archive contains the complete LaTeX source and compiled PDF, standalone checker, matched controls, proof construction and translation code, fixtures, test results, and a provenance record. The report can be rebuilt with two ordinary `pdflatex` invocations, without shell escape or network access. Run the supplied `reproduce.py` for the executable evidence. Use CPython 3.12.14 for the exact normalized-source measurements; the checker itself uses only standard-library facilities.

No repository branch, commit, issue, or pull request was written during this work. The existing formalization was inspected as source. The new metatheorems have not been represented as newly checked Lean results.

References

[1] V. Reshetnikov, *Concise presentations of intuitionistic primitive recursive arithmetic*, repository README, pinned commit above.
Pinned source on GitHub.

[2] *NativeSix.lean*: `SPrf`, `sprf_iff_nprf`, `SPrf.subst`; and *NativeDiagonal.lean*: diagonal guard and its completeness.
Pinned source on GitHub.

[3] *Raw clausal*, *reduced clause calculus*, and *uniform step* research packages, especially the clause-origin translation and ordinary-cut construction.
Pinned source on GitHub.

[4] *Mirror guard* research package, one-sided guarded formulas and their equation-induction lift.
Pinned source on GitHub.

[5] *Reflexive base* research package, particularly `proof.tex`: base-valid introduction and the substitution repair when a new reflexive base appears.
Pinned source on GitHub.

[6] *NativeGate.lean* and *NativeLogic.lean*: structural selectors and full intuitionistic logical macros.
Pinned source on GitHub.

[7] *Scalar.lean*, *TeqPair.lean*, and *PRAToScalar.lean*: precise scalar theory and primitive-recursive compilation.
Pinned source on GitHub.

[8] *BiInterpretation.lean* and *NativeBiInterpretation.lean*: verified scalar/native/iPRA interpretations, round trips, and equiconsistency.
Pinned source on GitHub.